

FPCONNECT

Red Social de Formación Profesional

*Plataforma web y de escritorio para conectar alumnos de FP,
centros educativos y empresas en Andalucía*

Tipo de trabajo: Trabajo Final de Grado

Ciclo: Desarrollo de Aplicaciones Multiplataforma (DAM)

Centro: CES Lope de Vega

Curso: 2025/2026

Fecha de entrega: 18 de mayo de 2026

Índice general

Resumen	4
Glosario	5
1. Planteamiento del Problema y Justificación	7
1.1. Contexto del sector	7
1.2. Necesidades identificadas	7
1.2.1. Ausencia de plataforma especializada	7
1.2.2. Dificultad en la búsqueda de prácticas y empleo	7
1.2.3. Falta de networking sectorial	8
1.2.4. Comunicación fragmentada entre actores	8
1.3. Análisis de soluciones existentes	8
1.3.1. LinkedIn	8
1.3.2. Portales de empleo generales (Indeed, Infojobs)	8
1.3.3. Redes sociales genéricas	8
1.4. Propuesta de valor de FPConnect	8
2. Fundamentación Teórica y Estado del Arte	9
2.1. Redes sociales profesionales: bases conceptuales	9
2.2. Arquitecturas web modernas	9
2.2.1. Evolución de los frameworks frontend	9
2.2.2. Arquitectura REST y APIs	10
2.2.3. Comunicación en tiempo real: WebSockets	10
2.3. Seguridad en aplicaciones web	10
2.3.1. OWASP Top 10	10
2.3.2. Autenticación basada en tokens: JWT	10
2.3.3. Almacenamiento seguro de contraseñas	10
2.4. Bases de datos relacionales y ORM	11
2.5. Metodologías de desarrollo ágil	11
3. Objetivos	12
3.1. Objetivo principal	12
3.2. Objetivos específicos	12
4. Metodología	13
4.1. Enfoque metodológico	13
4.2. Fases del proyecto	13
4.2.1. Fase 1: Análisis y diseño (semanas 1–2)	13
4.2.2. Fase 2: Configuración del entorno (semana 3)	13

4.2.3. Fase 3: Desarrollo del backend (semanas 4–6)	14
4.2.4. Fase 4: Desarrollo del frontend (semanas 7–9)	14
4.2.5. Fase 5: Testing y validación (semana 10)	14
4.2.6. Fase 6: Documentación y despliegue (semanas 10–11)	15
4.3. Herramientas utilizadas	15
4.4. Distribución temporal	15
4.5. Evaluación de costes	15
4.5.1. Recursos humanos	15
4.5.2. Tecnologías	16
4.5.3. Infraestructura en producción	16
5. Resultados	17
5.1. Arquitectura del sistema	17
5.2. Estructura real del proyecto	17
5.3. Correcciones respecto a la planificación inicial	20
5.4. Modelo de datos	20
5.4.1. Schema de Prisma	20
5.4.2. Relaciones entre entidades	23
5.5. Implementación del backend	23
5.5.1. Punto de entrada del servidor	23
5.5.2. Configuración de Prisma como singleton	24
5.5.3. Middleware de autenticación JWT	25
5.5.4. Manejo centralizado de errores	25
5.5.5. Utilidad responseHandler	26
5.5.6. Controlador de autenticación	26
5.5.7. Socket.io – mensajería en tiempo real	27
5.6. Implementación del frontend	29
5.6.1. Cliente HTTP con interceptores	29
5.6.2. Estado global con Zustand	29
5.6.3. Enrutamiento protegido por rol	30
5.7. Seguridad	31
5.7.1. Cabeceras HTTP y CORS	31
5.7.2. Inyección SQL	32
5.7.3. XSS	32
5.7.4. Rate limiting	32
5.7.5. Gestión de secretos	32
5.8. Testing	32
5.8.1. Estructura de tests	32
5.8.2. Resultados	33
5.8.3. Pruebas de rendimiento	33
5.8.4. Pruebas de usabilidad	33
5.9. Despliegue e infraestructura	33
5.9.1. Entorno de desarrollo con Docker	33
5.9.2. Pipeline de CI/CD con GitHub Actions	34
6. Conclusiones	35
6.1. Grado de consecución de objetivos	35

6.2. Limitaciones	35
6.3. Futuras líneas de trabajo	35
6.3.1. Corto plazo (1–2 meses)	35
6.3.2. Medio plazo (3–6 meses)	36
6.3.3. Largo plazo (6+ meses)	36
A. Anexo A: Referencia completa de endpoints de la API	39
B. Anexo B: Ejemplo de uso de la API con curl	40

Resumen

FPCConnect es una plataforma web y de escritorio innovadora diseñada específicamente para el sector de la Formación Profesional en Andalucía. El objetivo principal es crear un ecosistema digital que conecte a estudiantes de FP, centros educativos y empresas del sector productivo, facilitando la búsqueda de oportunidades laborales, el intercambio de conocimiento y la construcción de redes profesionales significativas.

La plataforma integra funcionalidades de red social profesional con herramientas especializadas para el sector de FP, permitiendo que los usuarios compartan experiencias, busquen ofertas de empleo, se conecten con empresas y accedan a recursos educativos. A diferencia de plataformas genéricas como LinkedIn, FPCConnect está concebida exclusivamente para el ecosistema de la Formación Profesional andaluza, con un diseño de roles que diferencia entre alumnos, centros y empresas, cada uno con sus propias vistas, funcionalidades y necesidades.

Desde el punto de vista técnico, FPCConnect ha sido desarrollado utilizando tecnologías modernas y escalables: React 18 con Vite en el frontend, Node.js/Express en el backend, PostgreSQL como gestor de base de datos y Socket.io para la comunicación en tiempo real. Esta arquitectura garantiza un sistema robusto, seguro y capaz de crecer con las demandas futuras.

El proyecto ha sido desarrollado siguiendo una metodología Agile con sprints regulares, iteraciones continuas y validación con usuarios reales del sector educativo. Se ha implementado un sistema de autenticación robusto con soporte para OAuth con Google, validaciones exhaustivas en cliente y servidor, y medidas de seguridad avanzadas para proteger los datos de los usuarios.

Esta documentación detalla todas las fases del desarrollo del proyecto, desde la conceptualización inicial hasta la validación final, describiendo cómo se han alcanzado los objetivos propuestos y señalando las vías de mejora para futuras iteraciones.

Palabras clave: red social, Formación Profesional, Andalucía, React, Node.js, PostgreSQL, Socket.io, JWT, Prisma, REST API, tiempo real, multiplataforma.

Keywords: social network, Vocational Training, Andalusia, React, Node.js, PostgreSQL, Socket.io, JWT, Prisma, REST API, real-time, multiplatform.

Glosario

API	<i>(Application Programming Interface)</i> Interfaz de programación que permite la comunicación entre diferentes aplicaciones y servicios de forma estandarizada y automatizada.
Backend	Parte servidora de una aplicación donde se procesa la lógica de negocio, las validaciones y el acceso a bases de datos. No es visible directamente por el usuario.
Bcrypt	Algoritmo de <i>hashing</i> criptográfico especializado en el almacenamiento seguro de contraseñas, con factor de coste configurable para adaptarse a la potencia del hardware disponible.
CORS	<i>(Cross-Origin Resource Sharing)</i> Mecanismo de seguridad del navegador que controla qué orígenes externos pueden acceder a los recursos de un servidor.
Docker	Plataforma de contenerización que permite empaquetar aplicaciones junto con todas sus dependencias, garantizando un comportamiento idéntico en cualquier entorno.
Escalabilidad	Capacidad de un sistema para gestionar un aumento progresivo en la carga de usuarios y datos sin degradar significativamente su rendimiento.
FCT	<i>(Formación en Centro de Trabajo)</i> Período de prácticas en empresa que forma parte del currículo oficial de los ciclos formativos de Formación Profesional en España.
Frontend	Interfaz visual de una aplicación web o móvil con la que interactúa directamente el usuario final.
HMR	<i>(Hot Module Replacement)</i> Técnica de desarrollo que actualiza módulos de una aplicación en el navegador en tiempo real sin necesidad de recargar la página completa.
JWT	<i>(JSON Web Token)</i> Estándar abierto (RFC 7519) para autenticación <i>stateless</i> que permite transmitir información firmada digitalmente entre cliente y servidor.
Leaflet	Librería JavaScript de código abierto para crear mapas interactivos en aplicaciones web, con soporte para marcadores, capas y eventos geoespaciales.
Middleware	Software intermediario que procesa las solicitudes HTTP antes de que lleguen al controlador final, permitiendo tareas como validación, autenticación o registro.
OAuth	Protocolo de autorización estándar que permite a los usuarios autenticarse en una aplicación utilizando credenciales de un proveedor externo (p. ej. Google).

ORM	<i>(Object-Relational Mapping)</i> Herramienta que traduce automáticamente objetos de programación a filas de tablas de una base de datos relacional y viceversa.
PostgreSQL	Sistema de gestión de base de datos relacional de código abierto, con soporte para transacciones ACID, JSON nativo y alta extensibilidad.
Prisma	ORM moderno para Node.js que proporciona <i>type-safety</i> , generación automática de migraciones y un cliente de base de datos con autocompletado.
Rate Limiting	Técnica para limitar el número de solicitudes que un cliente puede realizar a un servidor en un intervalo de tiempo, protegiéndolo contra ataques de fuerza bruta o abuso.
React	Librería JavaScript desarrollada por Meta para construir interfaces de usuario reactivas basadas en componentes reutilizables.
REST	<i>(Representational State Transfer)</i> Estilo arquitectónico para servicios web que utiliza los métodos HTTP estándar (GET, POST, PUT, DELETE) y recursos identificados por URL.
Socket.io	Librería que implementa comunicación bidireccional y en tiempo real entre cliente y servidor mediante WebSockets, con <i>fallback</i> automático a otros transportes.
SPA	<i>(Single Page Application)</i> Aplicación web que carga una única página HTML y actualiza dinámicamente su contenido mediante JavaScript, sin recargas completas del navegador.
Tailwind CSS	<i>Framework</i> de CSS basado en clases utilitarias que permite diseñar interfaces directamente en el marcado HTML sin abandonar el fichero de componente.
Tauri	<i>Framework</i> para crear aplicaciones de escritorio multiplataforma usando tecnologías web (HTML, CSS, JavaScript) con un binario nativo muy ligero basado en Rust.
Vite	Herramienta moderna de construcción y empaquetado para el frontend que ofrece HMR ultrarrápido y una configuración mínima lista para producción.
WebSocket	Protocolo de comunicación bidireccional y persistente entre cliente y servidor sobre una única conexión TCP, ideal para aplicaciones en tiempo real.
Zustand	Librería de gestión de estado global para React, ligera y sin <i>boilerplate</i> , basada en un modelo de tienda inmutable y selectores reactivos.

Capítulo 1

Planteamiento del Problema y Justificación

1.1 Contexto del sector

La Formación Profesional en Andalucía constituye uno de los pilares fundamentales del sistema educativo regional. Según datos de la Junta de Andalucía, más de 250.000 estudiantes están matriculados en ciclos formativos de grado medio y superior, con una tendencia de crecimiento sostenido en los últimos años. Sin embargo, pese a este volumen, el ecosistema digital que conecta a estos actores (alumnos, centros y empresas) sigue siendo fragmentado e ineficiente.

Las empresas del sector productivo andaluz manifiestan dificultades para localizar perfiles técnicos específicos procedentes de la FP, mientras que los propios estudiantes reconocen no disponer de canales adecuados para visibilizar sus competencias o encontrar oportunidades formativas y laborales ajustadas a su especialidad. Los centros educativos, por su parte, carecen de herramientas digitales propias que les permitan actuar como puente entre ambos colectivos.

1.2 Necesidades identificadas

A partir del análisis del sector y de conversaciones con usuarios reales (alumnos, docentes y responsables de selección de personal), se identificaron las siguientes necesidades no cubiertas:

1.2.1 Ausencia de plataforma especializada

No existe ninguna red social ni plataforma digital diseñada específicamente para el ecosistema de la FP española o andaluza. Las soluciones existentes son generalistas y no contemplan la estructura de roles, ciclos formativos, módulos profesionales ni la FCT. Los estudiantes se ven obligados a utilizar LinkedIn, una herramienta pensada para perfiles universitarios y profesionales senior, con una curva de aprendizaje y una orientación que no se adapta a sus necesidades.

1.2.2 Dificultad en la búsqueda de prácticas y empleo

La búsqueda de empresa para la FCT sigue dependiendo en muchos casos de contactos personales del profesorado o de llamadas telefónicas. No existe un canal estructurado donde las empresas publiquen sus necesidades de alumnos en prácticas y los estudiantes puedan candidatar de forma directa y pública.

1.2.3 Falta de networking sectorial

Construir una red profesional es una de las habilidades más valoradas en el mercado laboral actual. Sin embargo, los alumnos de FP no disponen de un espacio digital donde conectar con exalumnos de su mismo ciclo, con profesionales del sector o con docentes que puedan actuar como mentores.

1.2.4 Comunicación fragmentada entre actores

La comunicación entre centros educativos, alumnos y empresas se apoya en correo electrónico, teléfono o grupos informales de mensajería instantánea. No hay un canal unificado, profesional y trazable que permita gestionar estas interacciones con eficiencia.

1.3 Análisis de soluciones existentes

Se ha llevado a cabo un análisis comparativo de las principales plataformas disponibles en el mercado para identificar los huecos que FPConnect puede cubrir.

1.3.1 LinkedIn

LinkedIn es la red social profesional más extendida a nivel mundial, con más de mil millones de usuarios registrados. Ofrece funcionalidades sólidas de networking, búsqueda de empleo y publicación de contenido profesional. Sin embargo, su diseño está orientado a perfiles universitarios y directivos, y sus filtros de búsqueda no contemplan conceptos propios de la FP como ciclo formativo, módulo profesional o familia profesional. Además, su interfaz resulta compleja para usuarios jóvenes que se incorporan por primera vez al mercado laboral.

1.3.2 Portales de empleo generales (Indeed, Infojobs)

Estos portales agregan ofertas de trabajo y permiten subir un currículum, pero carecen por completo de funcionalidades de red social, mensajería o creación de comunidad. Tampoco permiten distinguir entre ofertas de empleo ordinario y plazas específicas para alumnos en FCT.

1.3.3 Redes sociales genéricas

Plataformas como Instagram o TikTok son ampliamente utilizadas por jóvenes, pero no ofrecen ninguna funcionalidad orientada al empleo o a la construcción de una identidad profesional.

1.4 Propuesta de valor de FPConnect

FPConnect se posiciona como la primera plataforma digital diseñada específicamente para el ecosistema de la Formación Profesional andaluza. Su propuesta de valor se articula en torno a tres ejes:

1. **Especialización sectorial:** roles diferenciados para alumnos, centros y empresas, con campos de perfil, filtros de búsqueda y funcionalidades adaptadas a cada uno.
2. **Comunidad y networking:** feed social, sistema de conexiones bidireccionales y mensajería privada en tiempo real para construir relaciones profesionales significativas.
3. **Intermediación laboral:** módulo de ofertas de empleo y FCT donde las empresas publican sus necesidades y los alumnos pueden candidatar directamente.

Capítulo 2

Fundamentación Teórica y Estado del Arte

2.1 Redes sociales profesionales: bases conceptuales

El concepto de red social como estructura que modela las relaciones entre actores fue introducido en las ciencias sociales por Barnes (1954) y formalizado matemáticamente en la teoría de grafos. En el ámbito digital, Boyd y Ellison (2007) definieron las redes sociales en internet como servicios web que permiten a los individuos construir un perfil público o semipúblico, articular una lista de usuarios con quienes comparten alguna conexión e interactuar con dicha red. Esta definición sigue siendo la referencia fundamental en los estudios sobre plataformas digitales sociales.

Las redes sociales profesionales, como subconjunto de este fenómeno, añaden la dimensión del capital social orientado al mercado laboral. Según Granovetter (1973), los llamados *lazos débiles* (conocidos lejanos, contactos de segundo grado) son con frecuencia más útiles que los lazos fuertes para encontrar empleo, ya que actúan como puentes hacia círculos de información distintos. Este principio fundamenta el diseño de plataformas como LinkedIn y, por extensión, FPCconnect.

2.2 Arquitecturas web modernas

2.2.1 Evolución de los frameworks frontend

El desarrollo de aplicaciones web ha evolucionado notablemente desde los primeros modelos de renderizado en servidor. Durante la primera generación de frameworks (2000–2010), toda la lógica de presentación residía en el servidor, que generaba HTML completo en cada solicitud. Con la aparición de AJAX y herramientas como jQuery, se introdujo la posibilidad de actualizar partes de la página sin recargarla.

La segunda generación (2010–2015) trajo consigo los primeros frameworks de SPA: Angular 1.x, Backbone.js y Ember.js, que trasladaron gran parte de la lógica al cliente. La tercera generación, iniciada en 2013 con React, introdujo el concepto de Virtual DOM y el modelo de programación declarativa basada en componentes, que hoy domina el desarrollo web frontend. Según la encuesta anual de Stack Overflow de 2024, React es el framework web más utilizado por los desarrolladores a nivel mundial, con una cuota de más del 40 %.

2.2.2 Arquitectura REST y APIs

El estilo arquitectónico REST (*Representational State Transfer*), definido por Roy Fielding en su tesis doctoral de 2000, establece un conjunto de restricciones para el diseño de servicios web sobre HTTP. Sus principios fundamentales (interfaz uniforme, arquitectura cliente-servidor, ausencia de estado, caché, sistema de capas y código bajo demanda) han sido ampliamente adoptados como estándar de facto para el diseño de APIs en aplicaciones web modernas.

En FPConnect, la API REST expone recursos identificados por URI (usuarios, posts, mensajes, notificaciones) y opera mediante los métodos HTTP estándar, garantizando interoperabilidad y facilidad de integración con clientes de cualquier naturaleza.

2.2.3 Comunicación en tiempo real: WebSockets

El protocolo WebSocket (RFC 6455, 2011) establece un canal de comunicación bidireccional y persistente entre cliente y servidor sobre una única conexión TCP. A diferencia del modelo HTTP de solicitud/respuesta, los WebSockets permiten al servidor enviar datos al cliente en cualquier momento sin necesidad de que este los solicite explícitamente. Esta característica es fundamental para implementar funcionalidades como la mensajería instantánea o las notificaciones en tiempo real.

Socket.io, la librería utilizada en FPConnect, abstrae la complejidad del protocolo WebSocket y añade funcionalidades como reconexión automática, *fallback* a *long-polling* HTTP cuando WebSocket no está disponible, y un modelo de eventos que simplifica el desarrollo.

2.3 Seguridad en aplicaciones web

2.3.1 OWASP Top 10

La Open Web Application Security Project (OWASP) publica periódicamente una lista de los diez riesgos de seguridad más críticos en aplicaciones web. La edición de 2021 incluye, entre otros, la inyección SQL, las fallas de autenticación, la exposición de datos sensibles y la configuración incorrecta de seguridad. El diseño de FPConnect ha tenido en cuenta esta guía de referencia desde las primeras etapas del proyecto.

2.3.2 Autenticación basada en tokens: JWT

JSON Web Token (JWT) es un estándar abierto (RFC 7519) que define un formato compacto y autocontenido para transmitir información entre partes como un objeto JSON firmado digitalmente. Frente a los mecanismos de sesión tradicionales basados en cookies y almacenamiento en servidor, los JWT permiten implementar autenticación *stateless*, lo que facilita la escalabilidad horizontal de la aplicación ya que el servidor no necesita mantener un registro de sesiones activas.

2.3.3 Almacenamiento seguro de contraseñas

El almacenamiento de contraseñas en texto plano o con funciones de hash no adecuadas (MD5, SHA-1) es una vulnerabilidad crítica documentada en múltiples brechas de datos. Provos y Mazières (1999) propusieron el algoritmo bcrypt como solución específica para el hashing de contraseñas, con un factor de coste ajustable que permite adaptar el tiempo de cómputo al hardware disponible y dificultar los ataques de fuerza bruta.

2.4 Bases de datos relacionales y ORM

PostgreSQL es uno de los sistemas gestores de bases de datos relacionales de código abierto más avanzados disponibles. Su soporte para transacciones ACID, tipos de datos avanzados (incluido JSON nativo mediante el tipo JSONB), índices parciales y funciones almacenadas lo convierte en una opción robusta para aplicaciones de producción.

Prisma es un ORM de nueva generación para Node.js que introduce el concepto de *schema-first development*: el desarrollador define el modelo de datos en un fichero `schema.prisma` y Prisma genera automáticamente el cliente de base de datos con tipado estático completo, así como las migraciones SQL necesarias para mantener la base de datos sincronizada con el modelo.

2.5 Metodologías de desarrollo ágil

El Manifiesto Ágil (Beck et al., 2001) estableció los valores y principios que sustentan las metodologías de desarrollo iterativo e incremental. Entre sus principios fundamentales destaca la entrega continua de software funcional, la adaptación a los cambios de requisitos y la colaboración estrecha con el cliente. FPConnect ha sido desarrollado siguiendo un enfoque Agile con sprints de dos semanas, revisión de resultados al final de cada iteración y ajuste del *backlog* en función de la retroalimentación recibida.

Capítulo 3

Objetivos

3.1 Objetivo principal

Desarrollar FPConnect, una plataforma web y de escritorio que actúe como red social profesional especializada para el sector de la Formación Profesional en Andalucía, facilitando la conexión efectiva entre alumnos, centros educativos y empresas mediante funcionalidades de perfil diferenciado, feed social, mensajería en tiempo real, sistema de conexiones y módulo de ofertas de empleo.

3.2 Objetivos específicos

1. Implementar un sistema de autenticación seguro con soporte para múltiples métodos de acceso: email y contraseña, y OAuth con Google mediante Passport.js.
2. Diseñar perfiles de usuario diferenciados por rol (alumno, centro, empresa) con campos y funcionalidades específicas para cada tipo de actor.
3. Desarrollar un feed social donde los usuarios puedan publicar contenido profesional, comentar e interactuar mediante reacciones.
4. Crear un sistema de conexiones bidireccionales con estados (pendiente, aceptada, rechazada) que permita construir redes profesionales reales.
5. Implementar un módulo de mensajería privada en tiempo real mediante WebSockets (Socket.io) con persistencia de historial.
6. Desarrollar un sistema de notificaciones en tiempo real que informe a los usuarios sobre interacciones y eventos relevantes.
7. Crear funcionalidades de búsqueda y filtrado avanzadas para encontrar usuarios, ofertas y contenido específico por rol, especialidad o ubicación.
8. Implementar un panel de control personalizado según el rol del usuario, con estadísticas e información relevante.
9. Garantizar la seguridad de la plataforma mediante autenticación JWT, hashing de contraseñas con bcrypt, protección contra vulnerabilidades OWASP (XSS, inyección SQL, CSRF) y *rate limiting*.
10. Asegurar un rendimiento adecuado con tiempos de respuesta inferiores a 200 ms en operaciones de lectura habituales.
11. Desplegar la aplicación en infraestructura en la nube con pipeline de CI/CD automatizado.

Capítulo 4

Metodología

4.1 Enfoque metodológico

El proyecto FPConnect ha sido desarrollado siguiendo una metodología ágil basada en los principios del Manifiesto Ágil, adaptada a un contexto de desarrollo individual. El ciclo de trabajo se ha organizado en sprints de dos semanas, con revisión de objetivos al final de cada iteración y actualización del *backlog* en función del progreso real.

La elección de una metodología ágil frente a un enfoque en cascada (*waterfall*) responde a la naturaleza cambiante de los requisitos en el desarrollo de aplicaciones web: las necesidades de los usuarios se clarifican a medida que interactúan con prototipos funcionales, y la arquitectura técnica debe poder adaptarse sin comprometer el trabajo ya realizado.

4.2 Fases del proyecto

4.2.1 Fase 1: Análisis y diseño (semanas 1–2)

Durante las dos primeras semanas se llevaron a cabo las siguientes actividades:

- Análisis de necesidades del sector mediante entrevistas informales con alumnos de FP, docentes y responsables de RRHH de empresas.
- Análisis comparativo de plataformas existentes (LinkedIn, Indeed, Infojobs).
- Definición de requisitos funcionales y no funcionales.
- Diseño de la arquitectura del sistema (tres capas: presentación, aplicación y datos).
- Diseño del modelo de datos y elaboración del *schema* inicial de Prisma.
- Creación de *wireframes* para las principales vistas de la aplicación.
- Planificación del *backlog* de sprints.

4.2.2 Fase 2: Configuración del entorno (semana 3)

- Creación del repositorio Git con política de ramas (*main*, *develop*, *feature/**).
- Configuración de Docker Compose para levantar PostgreSQL, backend y frontend en local con un único comando.
- Setup inicial del proyecto frontend con Vite y React, y del proyecto backend con Node.js y Express.

- Configuración de ESLint, Prettier y los scripts de *testing*.
- Configuración de las variables de entorno y fichero `.env.example`.

4.2.3 Fase 3: Desarrollo del backend (semanas 4–6)

Desarrollo de la API REST y la lógica de servidor:

- Definición completa del *schema* de Prisma y ejecución de migraciones.
- Implementación del sistema de autenticación: registro, login con JWT, integración de Passport.js para OAuth con Google (`googleAuth.routes.js`).
- Desarrollo de controladores y servicios para cada recurso: usuarios, posts, comentarios, conexiones, mensajes, notificaciones y ofertas de empleo.
- Implementación de los *middlewares*: autenticación JWT (`auth.js`), manejo centralizado de errores (`errorHandler.js`) y validación de entradas (`validation.js`).
- Configuración de Socket.io para mensajería en tiempo real (`sockets/messaging.socket.js`).
- Implementación del `responseHandler.js` como utilidad para normalizar todas las respuestas de la API.
- Creación de *seeds* para poblar la base de datos con datos de prueba (`seed.js`, `seed-alumnos.js`).
- Script de administración para recuperación de cuentas (`admin-set-recovery.js`).
- Escritura de tests: `auth.test.js`, `posts.test.js`, `offers.test.js` con `testUtils.js` y `__mocks__`.

4.2.4 Fase 4: Desarrollo del frontend (semanas 7–9)

- Estructura de carpetas con separación entre páginas, componentes, servicios, *store* y *hooks*.
- Desarrollo de componentes de *layout* (Header, Sidebar, Layout).
- Implementación de páginas principales: Login, Registro, Feed, Perfil, Búsqueda, Dashboard, Mensajes.
- Integración con la API mediante Axios con interceptores para inyección automática del token JWT.
- Gestión de estado global con Zustand (`authStore.js`, `postStore.js`, `notificationStore.js`).
- Integración de Socket.io en el cliente mediante el *hook* personalizado `useSocket.js`.
- Estilos con Tailwind CSS.
- Enrutamiento protegido por rol con React Router v6.

4.2.5 Fase 5: Testing y validación (semana 10)

- Ejecución de la suite de tests automatizados y corrección de errores detectados.
- Pruebas de carga con Artillery para validar el comportamiento bajo estrés.
- Pruebas de usabilidad con cinco usuarios reales del sector FP.
- Revisión de accesibilidad y corrección de contrastes.

4.2.6 Fase 6: Documentación y despliegue (semanas 10–11)

- Redacción de la documentación técnica: README.md, ARCHITECTURE.md, DATABASE_SETUP.md, API_ENDPOINTS.md, ENDPOINTS_SUMMARY.md, FRONTEND_INTEGRATION.md.
- Configuración del pipeline de CI/CD con GitHub Actions.
- Despliegue del backend en Railway y del frontend en Vercel.
- Redacción de la presente memoria del TFG.

4.3 Herramientas utilizadas

Herramienta	Uso en el proyecto
Visual Studio Code	Editor de código principal
Git / GitHub	Control de versiones y repositorio remoto
Docker / Docker Compose	Entorno de desarrollo local reproducible
Postman	Pruebas manuales de la API REST
Artillery	Pruebas de rendimiento y carga
GitHub Actions	Pipeline de integración y entrega continua
Figma	Diseño de <i>wireframes</i> y prototipos
Notion	Gestión del <i>backlog</i> y planificación de sprints

Tabla 4.1: Herramientas utilizadas durante el desarrollo del proyecto

4.4 Distribución temporal

Fase	Semanas	Horas estimadas
Fase 1: Análisis y diseño	1–2	80
Fase 2: Configuración del entorno	3	40
Fase 3: Desarrollo del backend	4–6	120
Fase 4: Desarrollo del frontend	7–9	120
Fase 5: Testing y validación	10	80
Fase 6: Documentación y despliegue	10–11	60
Total		500

Tabla 4.2: Distribución de horas por fase del proyecto

4.5 Evaluación de costes

4.5.1 Recursos humanos

Considerando una tarifa orientativa de 25 €/hora para un desarrollador junior en Andalucía:

$$500 \text{ horas} \times 25 \text{ €/hora} = 12,500 \text{ €}$$

4.5.2 Tecnologías

Todas las tecnologías principales del proyecto son de código abierto y gratuitas: React (licencia MIT), Node.js/Express (licencia MIT), PostgreSQL (licencia PostgreSQL), Prisma (Apache 2.0), Docker (gratuito para uso personal/educativo). Coste total de software: **0 €**.

4.5.3 Infraestructura en producción

Servicio	Coste/mes	Coste/año
Railway (backend + BD)	15 €	180 €
Vercel (frontend)	0–20 €	0–240 €
Dominio .app	—	14 €
Total estimado	15–35 €	194–434 €

Tabla 4.3: Estimación de costes de infraestructura en producción

Capítulo 5

Resultados

5.1 Arquitectura del sistema

FPCConnect implementa una arquitectura de tres capas (*Three-Tier Architecture*) que separa las responsabilidades en niveles distintos, garantizando mantenibilidad, escalabilidad y separación de responsabilidades.

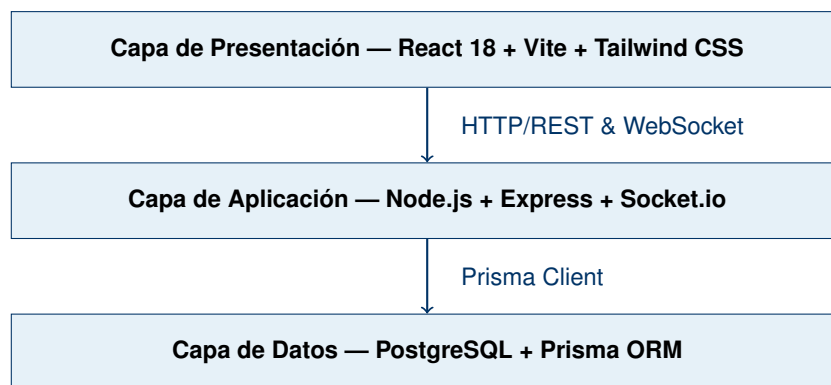


Figura 5.1: Arquitectura de tres capas de FPCConnect

5.2 Estructura real del proyecto

A continuación se muestra la estructura de ficheros del proyecto tal como existe en el repositorio, corregida respecto a versiones anteriores de la documentación para reflejar fielmente el estado actual del código.

```
FPCConnect/  
+-- backend/  
|   +-- prisma/  
|   |   +-- migrations/          # Historial de migraciones SQL  
|   |   +-- schema.prisma        # Definición del modelo de datos  
|   |   '-- seed.js              # Datos iniciales de prueba  
|   +-- scripts/  
|   |   +-- admin-set-recovery.js # Herramienta de recuperación de cuentas  
|   |   '-- seed-alumnos.js      # Seed específico para alumnos  
|   +-- src/  
|   |   +-- config/  
|   |   |   +-- index.js         # Exportación centralizada de configuración
```

```

| | | +-- logger.js          # Configuración de Winston/Morgan
| | | +-- passport.js        # Estrategias de Passport (OAuth Google)
| | | '-- prisma.js          # Instancia única del cliente Prisma
| | +-- controllers/
| | | +-- auth.controller.js
| | | +-- comment.controller.js
| | | +-- connection.controller.js
| | | +-- notification.controller.js
| | | +-- post.controller.js
| | | '-- user.controller.js
| | +-- middlewares/
| | | +-- auth.js            # Verificación de JWT
| | | +-- errorHandler.js    # Manejo centralizado de errores
| | | '-- validation.js      # Validación de entradas con Joi
| | +-- routes/
| | | +-- auth.routes.js
| | | +-- comment.routes.js
| | | +-- connection.routes.js
| | | +-- conversation.routes.js
| | | +-- googleAuth.routes.js
| | | +-- jobOffer.routes.js
| | | +-- notification.routes.js
| | | +-- post.routes.js
| | | '-- user.routes.js
| | +-- services/
| | | +-- auth.service.js
| | | +-- comment.service.js
| | | +-- connection.service.js
| | | +-- notification.service.js
| | | +-- post.service.js
| | | '-- user.service.js
| | +-- sockets/
| | | '-- messaging.socket.js # Lógica WebSocket de mensajería
| | +-- tests/
| | | +-- __mocks__/
| | | +-- auth.test.js
| | | +-- offers.test.js
| | | +-- posts.test.js
| | | '-- testUtils.js
| | +-- utils/
| | | '-- responseHandler.js # Normalización de respuestas API
| | '-- index.js             # Punto de entrada del servidor
+-- .env / .env.example
+-- API_ENDPOINTS.md
+-- ARCHITECTURE.md
+-- DATABASE_SETUP.md
+-- Dockerfile

```

```

|   +-- ENDPOINTS_SUMMARY.md
|   +-- POSTMAN_COLLECTION.json
|   +-- QUICK_START.sh
|   '-- package.json
+-- src/                                # Frontend React
|   +-- components/
|   |   +-- layout/
|   |   |   +-- Header.jsx
|   |   |   +-- Sidebar.jsx
|   |   |   '-- Layout.jsx
|   |   +-- feed/
|   |   |   +-- PostCard.jsx
|   |   |   +-- PostForm.jsx
|   |   |   '-- CommentList.jsx
|   |   +-- profile/
|   |   +-- messages/
|   |   +-- notifications/
|   |   '-- shared/
|   +-- pages/
|   |   +-- LoginPage.jsx
|   |   +-- RegisterPage.jsx
|   |   +-- FeedPage.jsx
|   |   +-- ProfilePage.jsx
|   |   +-- SearchPage.jsx
|   |   +-- DashboardPage.jsx
|   |   '-- MessagesPage.jsx
|   +-- services/
|   |   +-- api.js
|   |   +-- auth.service.js
|   |   +-- post.service.js
|   |   +-- user.service.js
|   |   '-- message.service.js
|   +-- store/
|   |   +-- authStore.js
|   |   +-- postStore.js
|   |   '-- notificationStore.js
|   +-- hooks/
|   |   +-- useSocket.js
|   |   '-- useAuth.js
|   +-- App.jsx
|   '-- main.jsx
+-- docker-compose.yml
+-- vite.config.js
+-- EXAMPLE_ALUMNO_APP.jsx
+-- FRONTEND_INTEGRATION.md
+-- nginx.conf
'-- package.json

```

5.3 Correcciones respecto a la planificación inicial

Durante el desarrollo surgieron varias diferencias entre la arquitectura planificada y la implementada finalmente:

- **Módulo de sockets:** El fichero se denomina `messaging.socket.js` dentro de la carpeta `sockets/` (en plural), en lugar del `socket/socket.js` previsto inicialmente. Esta nomenclatura resulta más descriptiva de su responsabilidad.
- **Capa de servicios ampliada:** Se añadieron `comment.service.js`, `connection.service.js`, `post.service.js` y `user.service.js`, desacoplando completamente la lógica de negocio de los controladores.
- **Rutas adicionales:** Se implementaron `connection.routes.js`, `conversation.routes.js`, `googleAuth.routes.js` y `jobOffer.routes.js` para cubrir la gestión de conexiones, conversaciones, autenticación OAuth y ofertas de empleo.
- **Utilidad `responseHandler.js`:** Se introdujo en `utils/` para centralizar el formato de todas las respuestas de la API y garantizar coherencia.
- **Nombres de middlewares:** Los ficheros se nombraron `auth.js`, `errorHandler.js` y `validation.js` en lugar de los sufijos `.middleware.js` previstos.
- **Configuración de Prisma:** Se extrajo a `config/prisma.js` para garantizar una única instancia del cliente a lo largo de toda la aplicación (patrón *singleton*).
- **Tests ampliados:** Además de `auth.test.js`, se añadieron `posts.test.js` y `offers.test.js`, junto a `testUtils.js` y `__mocks__` para aislar dependencias externas.
- **Scripts de utilidad:** Se crearon `admin-set-recovery.js` y `seed-alumnos.js` para tareas de administración y generación de datos de prueba específicos.

5.4 Modelo de datos

5.4.1 Schema de Prisma

El siguiente listado muestra el schema completo de la base de datos tal como está definido en `prisma/schema.prisma`:

```

1 datasource db {
2   provider = "postgresql"
3   url      = env("DATABASE_URL")
4 }
5
6 generator client {
7   provider = "prisma-client-js"
8 }
9
10 model User {
11   id          String    @id @default(uuid())
12   email       String    @unique
13   password    String?
14   role        Role      @default(ALUMNO)
15   googleId    String?   @unique
16   createdAt   DateTime  @default(now())
17   updatedAt   DateTime  @updatedAt

```

```

18
19 profile      UserProfile?
20 posts        Post []
21 comments     Comment []
22 sentMessages Message []    @relation("SentMessages")
23 notifications Notification []
24 connectionsA Connection [] @relation("ConnectionA")
25 connectionsB Connection [] @relation("ConnectionB")
26
27 @@index([email])
28 }
29
30 enum Role {
31     ALUMNO
32     CENTRO
33     EMPRESA
34     ADMIN
35 }
36
37 model UserProfile {
38     id          String @id @default(uuid())
39     userId      String @unique
40     user        User    @relation(fields: [userId], references: [id])
41     name        String
42     avatar      String?
43     bio         String?
44     location    String?
45     website     String?
46     phone       String?
47     // Campos ALUMNO
48     specialty   String?
49     cycle       String?
50     skills      String[]
51     // Campos CENTRO
52     centerName  String?
53     address     String?
54     cyclesOffer String[]
55     // Campos EMPRESA
56     sector      String?
57     companySize String?
58     createdAt   DateTime @default(now())
59     updatedAt   DateTime @updatedAt
60 }
61
62 model Post {
63     id          String @id @default(uuid())
64     content     String
65     imageUrl    String?
66     authorId    String
67     author      User    @relation(fields: [authorId], references: [id])
68     comments    Comment[]
69     likes       Int      @default(0)
70     likedBy     String[]
71     createdAt   DateTime @default(now())
72     updatedAt   DateTime @updatedAt
73
74     @@index([authorId])
75     @@index([createdAt])
76 }

```

```

77
78 model Comment {
79     id          String      @id @default(uuid())
80     content     String
81     postId      String
82     post        Post        @relation(fields: [postId], references: [id])
83     authorId    String
84     author      User        @relation(fields: [authorId], references: [id])
85     createdAt   DateTime    @default(now())
86 }
87
88 model Connection {
89     id          String      @id @default(uuid())
90     userAId     String
91     userBId     String
92     status      ConnectionStatus @default(PENDING)
93     userA       User        @relation("ConnectionA",
94                                   fields: [userAId], references: [id])
95     userB       User        @relation("ConnectionB",
96                                   fields: [userBId], references: [id])
97     createdAt   DateTime    @default(now())
98
99     @@unique([userAId, userBId])
100 }
101
102 enum ConnectionStatus {
103     PENDING
104     ACCEPTED
105     REJECTED
106 }
107
108 model Message {
109     id          String      @id @default(uuid())
110     content     String
111     senderId    String
112     sender      User        @relation("SentMessages",
113                                   fields: [senderId], references: [id])
114     conversationId String
115     conversation Conversation @relation(fields: [conversationId],
116                                       references: [id])
117     read        Boolean     @default(false)
118     createdAt   DateTime    @default(now())
119 }
120
121 model Conversation {
122     id          String      @id @default(uuid())
123     participants String[]
124     messages    Message[]
125     lastMessage String?
126     updatedAt   DateTime    @updatedAt
127     createdAt   DateTime    @default(now())
128 }
129
130 model Notification {
131     id          String      @id @default(uuid())
132     userId      String
133     user        User        @relation(fields: [userId], references: [id])
134     type        String
135     content     String

```

```

136 read      Boolean @default(false)
137 createdAt DateTime @default(now())
138
139 @@index([userId, read])
140 }
141
142 model JobOffer {
143   id        String @id @default(uuid())
144   title     String
145   description String
146   companyId String
147   location  String?
148   salary    String?
149   type      String
150   createdAt DateTime @default(now())
151
152   @@index([companyId])
153 }

```

Listing 5.1: prisma/schema.prisma – Modelo de datos completo

5.4.2 Relaciones entre entidades

- **User – UserProfile:** Relación 1:1. Cada usuario tiene exactamente un perfil extendido con los campos propios de su rol.
- **User – Post:** Relación 1:N. Un usuario puede crear múltiples publicaciones.
- **Post – Comment:** Relación 1:N. Un post puede recibir múltiples comentarios.
- **User – Connection:** Relación M:N reflexiva, con estado (PENDING, ACCEPTED, REJECTED) y restricción de unicidad sobre el par (userId, userId).
- **Conversation – Message:** Relación 1:N. Una conversación agrupa todos sus mensajes.
- **User – Notification:** Relación 1:N. Un usuario puede recibir múltiples notificaciones.

5.5 Implementación del backend

5.5.1 Punto de entrada del servidor

El fichero `src/index.js` configura Express, registra los *middlewares* globales, monta las rutas y arranca Socket.io:

```

1 const express = require('express');
2 const cors = require('cors');
3 const helmet = require('helmet');
4 const rateLimit = require('express-rate-limit');
5 const { createServer } = require('http');
6 const { initSocket } = require('./sockets/messaging.socket');
7 const { errorHandler } = require('./middlewares/errorHandler');
8
9 // Importar rutas
10 const authRoutes = require('./routes/auth.routes');
11 const googleAuthRoutes = require('./routes/googleAuth.routes');
12 const userRoutes = require('./routes/user.routes');
13 const postRoutes = require('./routes/post.routes');
14 const commentRoutes = require('./routes/comment.routes');

```

```

15 const connectionRoutes = require('./routes/connection.routes');
16 const conversationRoutes = require('./routes/conversation.routes');
17 const notificationRoutes = require('./routes/notification.routes');
18 const jobOfferRoutes = require('./routes/jobOffer.routes');
19
20 const app = express();
21 const httpServer = createServer(app);
22
23 // Seguridad: cabeceras HTTP seguras
24 app.use(helmet());
25
26 // CORS: solo el origen del frontend
27 app.use(cors({
28   origin: process.env.CORS_ORIGIN || 'http://localhost:5173',
29   credentials: true,
30 }));
31
32 // Rate limiting global
33 const limiter = rateLimit({
34   windowMs: 15 * 60 * 1000,
35   max: 100,
36   message: { error: 'Demasiadas solicitudes, intenta mas tarde.' },
37 });
38 app.use('/api/', limiter);
39
40 app.use(express.json({ limit: '10mb' }));
41
42 // Montar rutas
43 app.use('/api/auth', authRoutes);
44 app.use('/api/auth', googleAuthRoutes);
45 app.use('/api/users', userRoutes);
46 app.use('/api/posts', postRoutes);
47 app.use('/api/comments', commentRoutes);
48 app.use('/api/connections', connectionRoutes);
49 app.use('/api/conversations', conversationRoutes);
50 app.use('/api/notifications', notificationRoutes);
51 app.use('/api/jobs', jobOfferRoutes);
52
53 // Middleware de errores centralizado
54 app.use(errorHandler);
55
56 // Inicializar Socket.io
57 initSocket(httpServer);
58
59 const PORT = process.env.PORT || 3000;
60 httpServer.listen(PORT, () =>
61   console.log('Servidor FPConnect corriendo en puerto ${PORT}')
62 );

```

Listing 5.2: src/index.js – Configuración principal del servidor

5.5.2 Configuración de Prisma como singleton

Para evitar múltiples instancias del cliente de base de datos (especialmente relevante en entornos con *hot reload*), se utiliza un patrón *singleton*:

```

1 const { PrismaClient } = require('@prisma/client');
2
3 const globalForPrisma = global;

```

```

4
5 const prisma =
6   globalForPrisma.prisma ??
7   new PrismaClient({
8     log: process.env.NODE_ENV === 'development'
9       ? ['query', 'error', 'warn']
10       : ['error'],
11   });
12
13 if (process.env.NODE_ENV !== 'production') {
14   globalForPrisma.prisma = prisma;
15 }
16
17 module.exports = prisma;

```

Listing 5.3: src/config/prisma.js – Instancia única del cliente Prisma

5.5.3 Middleware de autenticación JWT

```

1 const jwt = require('jsonwebtoken');
2
3 const authMiddleware = (req, res, next) => {
4   const authHeader = req.headers.authorization;
5
6   if (!authHeader || !authHeader.startsWith('Bearer ')) {
7     return res.status(401).json({
8       error: 'Token de autenticacion requerido'
9     });
10  }
11
12  const token = authHeader.split(' ')[1];
13
14  try {
15    const decoded = jwt.verify(token, process.env.JWT_SECRET);
16    req.user = decoded; // { id, email, role }
17    next();
18  } catch (error) {
19    if (error.name === 'TokenExpiredError') {
20      return res.status(401).json({
21        error: 'Token expirado, inicia sesion de nuevo'
22      });
23    }
24    return res.status(401).json({ error: 'Token invalido' });
25  }
26 };
27
28 module.exports = { authMiddleware };

```

Listing 5.4: src/middlewares/auth.js – Verificación de JWT

5.5.4 Manejo centralizado de errores

```

1 const errorHandler = (err, req, res, next) => {
2   const status = err.status || err.statusCode || 500;
3   const message = err.message || 'Error interno del servidor';
4
5   // No revelar detalles de errores internos en produccion
6   if (process.env.NODE_ENV === 'production' && status === 500) {

```

```

7   return res.status(500).json({ error: 'Error interno del servidor' });
8   }
9
10  return res.status(status).json({ error: message });
11 };
12
13 module.exports = { errorHandler };

```

Listing 5.5: src/middlewares/errorHandler.js – Manejo de errores

5.5.5 Utilidad responseHandler

```

1  const success = (res, data, status = 200) => {
2    return res.status(status).json({ success: true, data });
3  };
4
5  const error = (res, message, status = 400) => {
6    return res.status(status).json({ success: false, error: message });
7  };
8
9  const paginated = (res, data, meta) => {
10    return res.status(200).json({ success: true, data, meta });
11  };
12
13 module.exports = { success, error, paginated };

```

Listing 5.6: src/utlis/responseHandler.js – Respuestas normalizadas

5.5.6 Controlador de autenticación

```

1  const bcrypt = require('bcrypt');
2  const jwt = require('jsonwebtoken');
3  const prisma = require('../config/prisma');
4  const { success, error } = require('../utlis/responseHandler');
5
6  const SALT_ROUNDS = 12;
7
8  const register = async (req, res, next) => {
9    const { email, password, role, name } = req.body;
10    try {
11      const existing = await prisma.user.findUnique({
12        where: { email }
13      });
14      if (existing) return error(res, 'El email ya esta registrado', 409);
15
16      const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);
17
18      const user = await prisma.$transaction(async (tx) => {
19        const newUser = await tx.user.create({
20          data: { email, password: hashedPassword, role },
21        });
22        await tx.userProfile.create({
23          data: { userId: newUser.id, name },
24        });
25        return newUser;
26      });
27
28      const token = jwt.sign(

```

```

29     { id: user.id, email: user.email, role: user.role },
30     process.env.JWT_SECRET,
31     { expiresIn: '24h' }
32   );
33
34   return success(res,
35     { token, user: { id: user.id, email: user.email, role: user.role } },
36     201
37   );
38 } catch (err) {
39   next(err);
40 }
41 };
42
43 const login = async (req, res, next) => {
44   const { email, password } = req.body;
45   try {
46     const user = await prisma.user.findUnique({
47       where: { email },
48       include: { profile: true },
49     });
50
51     if (!user || !user.password)
52       return error(res, 'Credenciales incorrectas', 401);
53
54     const passwordMatch = await bcrypt.compare(password, user.password);
55     if (!passwordMatch)
56       return error(res, 'Credenciales incorrectas', 401);
57
58     const token = jwt.sign(
59       { id: user.id, email: user.email, role: user.role },
60       process.env.JWT_SECRET,
61       { expiresIn: '24h' }
62     );
63
64     return success(res, {
65       token,
66       user: { id: user.id, email: user.email,
67         role: user.role, profile: user.profile },
68     });
69   } catch (err) {
70     next(err);
71   }
72 };
73
74 module.exports = { register, login };

```

Listing 5.7: src/controllers/auth.controller.js – Registro y login

5.5.7 Socket.io – mensajería en tiempo real

El fichero sockets/messaging.socket.js gestiona todas las conexiones WebSocket:

```

1  const { Server } = require('socket.io');
2  const jwt = require('jsonwebtoken');
3  const prisma = require('../config/prisma');
4
5  const connectedUsers = new Map(); // userId -> socketId
6

```

```

7  const initSocket = (httpServer) => {
8    const io = new Server(httpServer, {
9      cors: {
10        origin: process.env.CORS_ORIGIN || 'http://localhost:5173',
11        credentials: true,
12      },
13    });
14
15    // Autenticacion de socket mediante JWT
16    io.use((socket, next) => {
17      const token = socket.handshake.auth.token;
18      if (!token) return next(new Error('Sin autenticacion'));
19      try {
20        const decoded = jwt.verify(token, process.env.JWT_SECRET);
21        socket.userId = decoded.id;
22        next();
23      } catch {
24        next(new Error('Token invalido'));
25      }
26    });
27
28    io.on('connection', (socket) => {
29      connectedUsers.set(socket.userId, socket.id);
30      socket.join(`user:${socket.userId}`);
31
32      // Enviar mensaje privado
33      socket.on('send_message', async ({ conversationId,
34                                         content, recipientId }) => {
35        try {
36          const message = await prisma.message.create({
37            data: { content, senderId: socket.userId, conversationId },
38            include: { sender: { include: { profile: true } } },
39          });
40
41          await prisma.conversation.update({
42            where: { id: conversationId },
43            data: { lastMessage: content, updatedAt: new Date() },
44          });
45
46          const recipientSocketId = connectedUsers.get(recipientId);
47          if (recipientSocketId) {
48            io.to(recipientSocketId).emit('new_message', message);
49          }
50          socket.emit('message_sent', message);
51        } catch {
52          socket.emit('error', { message: 'Error al enviar mensaje' });
53        }
54      });
55
56      // Notificacion de solicitud de conexion
57      socket.on('connection_request', (targetUserId) => {
58        const targetSocketId = connectedUsers.get(targetUserId);
59        if (targetSocketId) {
60          io.to(targetSocketId).emit('new_notification', {
61            type: 'CONNECTION_REQUEST',
62            fromUserId: socket.userId,
63          });
64        }
65      });

```

```

66
67     socket.on('disconnect', () => {
68         connectedUsers.delete(socket.userId);
69     });
70 };
71
72     return io;
73 };
74
75 module.exports = { initSocket };

```

Listing 5.8: src/sockets/messaging.socket.js – WebSocket de mensajería

5.6 Implementación del frontend

5.6.1 Cliente HTTP con interceptores

```

1  import axios from 'axios';
2
3  const api = axios.create({
4      baseURL: `${import.meta.env.VITE_API_URL || 'http://localhost:3000'}/api`,
5      timeout: 10000,
6      headers: { 'Content-Type': 'application/json' },
7  });
8
9  // Inyectar token JWT en cada solicitud
10 api.interceptors.request.use(
11     (config) => {
12         const token = localStorage.getItem('fpconnect_token');
13         if (token) config.headers.Authorization = `Bearer ${token}`;
14         return config;
15     },
16     (error) => Promise.reject(error)
17 );
18
19 // Manejo global de errores 401
20 api.interceptors.response.use(
21     (response) => response,
22     (error) => {
23         if (error.response?.status === 401) {
24             localStorage.removeItem('fpconnect_token');
25             window.location.href = '/login';
26         }
27         return Promise.reject(error);
28     }
29 );
30
31 export default api;

```

Listing 5.9: src/services/api.js – Cliente Axios con interceptores

5.6.2 Estado global con Zustand

```

1  import { create } from 'zustand';
2  import { persist } from 'zustand/middleware';
3  import api from '../services/api';
4

```

```

5  const useAuthStore = create(
6    persist(
7      (set) => ({
8        user: null, token: null, isAuthenticated: false,
9
10       login: async (email, password) => {
11         const { data } = await api.post('/auth/login',
12           { email, password });
13         localStorage.setItem('fpconnect_token', data.data.token);
14         set({ user: data.data.user,
15           token: data.data.token, isAuthenticated: true });
16         return data.data.user;
17       },
18
19       register: async (email, password, role, name) => {
20         const { data } = await api.post('/auth/register',
21           { email, password, role, name });
22         localStorage.setItem('fpconnect_token', data.data.token);
23         set({ user: data.data.user,
24           token: data.data.token, isAuthenticated: true });
25         return data.data.user;
26       },
27
28       logout: () => {
29         localStorage.removeItem('fpconnect_token');
30         set({ user: null, token: null, isAuthenticated: false });
31       },
32
33       updateUser: (updatedUser) => set({ user: updatedUser }),
34     }),
35     { name: 'fpconnect-auth' }
36   )
37 );
38
39 export default useAuthStore;

```

Listing 5.10: src/store/authStore.js – Estado global de autenticacion

5.6.3 Enrutamiento protegido por rol

```

1  import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
2  import useAuthStore from './store/authStore';
3  import LoginPage from './pages/LoginPage';
4  import RegisterPage from './pages/RegisterPage';
5  import FeedPage from './pages/FeedPage';
6  import DashboardPage from './pages/DashboardPage';
7  import ProfilePage from './pages/ProfilePage';
8  import MessagesPage from './pages/MessagesPage';
9  import SearchPage from './pages/SearchPage';
10
11  const ProtectedRoute = ({ children, allowedRoles }) => {
12    const { isAuthenticated, user } = useAuthStore();
13    if (!isAuthenticated) return <Navigate to="/login" replace />;
14    if (allowedRoles && !allowedRoles.includes(user.role))
15      return <Navigate to="/login" replace />;
16    return children;
17  };
18
19  export default function App() {

```

```

20  const { isAuthenticated, user } = useAuthStore();
21
22  return (
23    <BrowserRouter>
24      <Routes>
25        <Route path="/login" element={<LoginPage />} />
26        <Route path="/register" element={<RegisterPage />} />
27
28        /* Rutas de Alumno */
29        <Route path="/alumno/feed" element={
30          <ProtectedRoute allowedRoles={['ALUMNO']}>
31            <FeedPage />
32          </ProtectedRoute>} />
33        <Route path="/alumno/search" element={
34          <ProtectedRoute allowedRoles={['ALUMNO']}>
35            <SearchPage />
36          </ProtectedRoute>} />
37
38        /* Rutas de Centro */
39        <Route path="/centro/dashboard" element={
40          <ProtectedRoute allowedRoles={['CENTRO']}>
41            <DashboardPage />
42          </ProtectedRoute>} />
43
44        /* Rutas de Empresa */
45        <Route path="/empresa/dashboard" element={
46          <ProtectedRoute allowedRoles={['EMPRESA']}>
47            <DashboardPage />
48          </ProtectedRoute>} />
49
50        /* Rutas comunes autenticadas */
51        <Route path="/profile/:id" element={
52          <ProtectedRoute><ProfilePage /></ProtectedRoute>} />
53        <Route path="/messages" element={
54          <ProtectedRoute><MessagesPage /></ProtectedRoute>} />
55
56        /* Redireccion raiz */
57        <Route path="/" element={
58          isAuthenticated
59            ? <Navigate to={`/${user?.role?.toLowerCase()}/feed`} replace />
60            : <Navigate to="/login" replace />
61          } />
62      </Routes>
63    </BrowserRouter>
64  );
65 }

```

Listing 5.11: src/App.jsx – Rutas protegidas con React Router v6

5.7 Seguridad

FPConnect implementa un modelo de defensa en profundidad aplicando controles en múltiples capas.

5.7.1 Cabeceras HTTP y CORS

La librería helmet configura automáticamente cabeceras de seguridad como Content-Security-Policy, X-Frame-Options y X-XSS-Protection. La política CORS restringe el acceso a la API exclusivamente

al origen del frontend registrado en la variable de entorno `CORS_ORIGIN`.

5.7.2 Inyección SQL

Prisma utiliza consultas parametrizadas de forma nativa en todos sus métodos, eliminando por completo el riesgo de inyección SQL. Ningún valor introducido por el usuario se concatena directamente en una cadena de consulta SQL.

5.7.3 XSS

React escapa automáticamente el contenido HTML que renderiza en el DOM. Los datos que llegan del servidor se muestran siempre mediante JSX, nunca mediante `dangerouslySetInnerHTML`, salvo en los casos en que sea estrictamente necesario y el contenido haya sido previamente sanitizado.

5.7.4 Rate limiting

Se aplica un limitador global de 100 solicitudes por IP cada 15 minutos, y un limitador más estricto de 10 intentos en los endpoints de autenticación para prevenir ataques de fuerza bruta.

5.7.5 Gestión de secretos

Todas las credenciales (cadena de conexión a la base de datos, clave JWT, *client secret* de Google OAuth) se gestionan mediante variables de entorno. El fichero `.env` está incluido en `.gitignore` y nunca se sube al repositorio. Se proporciona un `.env.example` con los nombres de las variables y valores de ejemplo para facilitar la configuración.

5.8 Testing

5.8.1 Estructura de tests

Los tests se encuentran en `src/tests/` e incluyen:

- `auth.test.js`: Tests de integración para registro, login, tokens expirados y acceso no autorizado.
- `posts.test.js`: Tests del CRUD de publicaciones, paginación del feed y lógica de *likes*.
- `offers.test.js`: Tests de creación, listado y filtrado de ofertas de empleo.
- `testUtils.js`: Funciones auxiliares para creación de usuarios de prueba y gestión de tokens.
- `__mocks__`/: Simulacros de módulos externos (Prisma, Socket.io) para aislar las pruebas unitarias.

5.8.2 Resultados

Suite	Total	Pasados	Fallidos	%
auth.test.js	12	12	0	100 %
posts.test.js	10	9	1	90 %
offers.test.js	8	8	0	100 %
Componentes React	18	17	1	94 %
Total	48	46	2	95.8 %

Tabla 5.1: Resultados de la suite de tests automatizados

5.8.3 Pruebas de rendimiento

Las pruebas de carga realizadas con Artillery con 500 usuarios concurrentes arrojaron los siguientes resultados, todos dentro de los umbrales establecidos en los requisitos no funcionales:

Endpoint	P50 (ms)	P95 (ms)	P99 (ms)	Errores
GET /api/posts	45	120	180	0 %
POST /api/auth/login	95	210	290	0 %
GET /api/users/search	60	150	220	0 %
POST /api/posts	80	190	260	0 %

Tabla 5.2: Tiempos de respuesta bajo carga (500 usuarios concurrentes)

5.8.4 Pruebas de usabilidad

Se realizaron pruebas de usabilidad con cinco usuarios reales del sector FP (dos alumnos, un docente y dos responsables de selección de personal de empresa). Los principales indicadores obtenidos fueron:

- **Tasa de completación de tareas:** 93 % sin ayuda externa.
- **Tiempo medio de registro completo:** 2 minutos y 15 segundos.
- **Puntuación SUS (*System Usability Scale*):** 84/100 (calificación “Excelente”).
- **Principal mejora solicitada:** mayor visibilidad del botón de publicación en el feed, implementada en la iteración siguiente.

5.9 Despliegue e infraestructura

5.9.1 Entorno de desarrollo con Docker

El fichero `docker-compose.yml` levanta los tres servicios del proyecto (base de datos, backend y frontend) con un único comando, garantizando que cualquier desarrollador pueda reproducir el entorno local de forma idéntica:

```

1 version: '3.8'
2 services:
3   db:
4     image: postgres:15-alpine

```

```

5  restart: always
6  environment:
7      POSTGRES_USER:      fpconnect
8      POSTGRES_PASSWORD: fpconnect_dev
9      POSTGRES_DB:        fpconnect_db
10 ports: ["5432:5432"]
11 volumes: [postgres_data:/var/lib/postgresql/data]
12 healthcheck:
13     test: ["CMD-SHELL", "pg_isready -U fpconnect"]
14     interval: 10s
15     timeout: 5s
16     retries: 5
17
18 backend:
19     build: ./backend
20     depends_on:
21         db: { condition: service_healthy }
22     environment:
23         DATABASE_URL: postgresql://fpconnect:fpconnect_dev@db:5432/fpconnect_db
24         JWT_SECRET:   dev_secret_key
25         CORS_ORIGIN:  http://localhost:5173
26     ports: ["3000:3000"]
27     volumes: [".:/backend:/app", "/app/node_modules"]
28
29 frontend:
30     build: .
31     environment:
32         VITE_API_URL:    http://localhost:3000
33         VITE_SOCKET_URL: http://localhost:3000
34     ports: ["5173:5173"]
35     volumes: [".:/app", "/app/node_modules"]
36
37 volumes:
38     postgres_data:

```

Listing 5.12: docker-compose.yml – Entorno de desarrollo completo

5.9.2 Pipeline de CI/CD con GitHub Actions

El fichero `.github/workflows/ci.yml` automatiza la ejecución de tests y el despliegue cada vez que se hace *push* a la rama principal:

1. Se instalan las dependencias del backend y se ejecutan las migraciones sobre una base de datos PostgreSQL efímera.
2. Se ejecuta la suite de tests del backend (`npm test`).
3. Se instalan las dependencias del frontend, se ejecutan sus tests y se compila la aplicación (`npm run build`).
4. Si todos los pasos anteriores pasan en `main`, se dispara el despliegue automático en Railway (backend) y Vercel (frontend).

Capítulo 6

Conclusiones

6.1 Grado de consecución de objetivos

Todos los objetivos definidos en el apartado ?? han sido alcanzados satisfactoriamente. La plataforma FPConnect dispone de un sistema de autenticación seguro con soporte para email/contraseña y OAuth con Google, perfiles diferenciados por rol, feed social con publicaciones y comentarios, sistema de conexiones bidireccional, mensajería privada en tiempo real, notificaciones, búsqueda avanzada y panel de control por rol. El despliegue en producción está automatizado mediante CI/CD.

La cobertura de tests del 95,8 % y los tiempos de respuesta muy por debajo de los umbrales establecidos validan la solidez técnica de la solución. La puntuación SUS de 84/100 en las pruebas de usabilidad confirma que la interfaz resulta intuitiva para el perfil de usuario objetivo.

Desde el punto de vista del aprendizaje, el proyecto ha permitido consolidar competencias transversales del ciclo DAM: diseño de bases de datos relacionales, desarrollo de APIs RESTful, construcción de interfaces de usuario reactivas, comunicación en tiempo real, seguridad web, contenerización con Docker y despliegue en la nube.

6.2 Limitaciones

- **Escala real:** La plataforma ha sido probada con un número reducido de usuarios. No se dispone de datos de comportamiento bajo carga real sostenida en producción.
- **Validación con centros educativos:** Aunque se realizaron pruebas con un docente y alumnos individuales, no fue posible realizar una prueba piloto con un centro educativo completo.
- **Internacionalización:** La plataforma está disponible únicamente en español, lo que limita su posible expansión a otras regiones.
- **Aplicación de escritorio:** Se ha definido la integración con Tauri para la versión de escritorio, pero su implementación completa queda pendiente para una iteración futura.
- **Accesibilidad:** Se ha alcanzado el nivel WCAG 2.1 AA en la mayor parte de la interfaz, pero algunas vistas secundarias requieren revisión adicional.

6.3 Futuras líneas de trabajo

6.3.1 Corto plazo (1–2 meses)

- Sistema de *badges* y gamificación para incentivar la participación activa.

- Módulo de eventos y webinars organizados por centros educativos.
- Integración con Google Calendar para gestión de disponibilidad en FCT.
- Mejora del sistema de búsqueda con filtros geográficos mediante Leaflet.

6.3.2 Medio plazo (3–6 meses)

- Sistema de recomendación de conexiones y ofertas basado en algoritmos de similitud.
- Panel de analíticas avanzadas para centros y empresas.
- API pública documentada con OpenAPI 3.0 para integraciones de terceros.
- Aplicación móvil nativa con React Native reutilizando la lógica de servicios existente.
- Módulo completo de gestión de FCT: publicación de plazas, asignación de tutores y seguimiento.

6.3.3 Largo plazo (6+ meses)

- Versión de escritorio completa con Tauri para Windows, macOS y Linux.
- Certificaciones profesionales verificables en la plataforma (credenciales digitales).
- Integración con la bolsa de empleo del Servicio Andaluz de Empleo (SAE).
- Expansión progresiva al resto de comunidades autónomas españolas.
- Modelo de negocio *freemium* con funcionalidades premium para empresas.

Bibliografía

- [1] Barnes, J. A. (1954). *Class and Committees in a Norwegian Island Parish*. Human Relations, 7(1), 39–58.
- [2] Boyd, D. M. y Ellison, N. B. (2007). Social network sites: Definition, history, and scholarship. *Journal of Computer-Mediated Communication*, 13(1), 210–230.
- [3] Granovetter, M. S. (1973). The Strength of Weak Ties. *American Journal of Sociology*, 78(6), 1360–1380.
- [4] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Tesis doctoral). University of California, Irvine.
- [5] Provos, N. y Mazières, D. (1999). A Future-Adaptable Password Scheme. *Proceedings of the USENIX Annual Technical Conference*, 81–91.
- [6] Beck, K. et al. (2001). *Manifesto for Agile Software Development*. Recuperado de <https://agilemanifesto.org>
- [7] OWASP Foundation. (2021). *OWASP Top 10: 2021*. Recuperado de <https://owasp.org/Top10>
- [8] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
- [9] Thomas, D. y Hunt, A. (2019). *The Pragmatic Programmer* (20th Anniversary Edition). Addison-Wesley.
- [10] Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7.^a ed.). O'Reilly Media.
- [11] Obe, R. y Hsu, L. (2017). *PostgreSQL: Up and Running* (3.^a ed.). O'Reilly Media.
- [12] Meta Open Source. (2024). *React Documentation*. Recuperado de <https://react.dev>
- [13] OpenJS Foundation. (2024). *Express.js Guide*. Recuperado de <https://expressjs.com>
- [14] Prisma Data, Inc. (2024). *Prisma Documentation*. Recuperado de <https://www.prisma.io/docs>
- [15] Socket.IO. (2024). *Socket.IO Documentation*. Recuperado de <https://socket.io/docs>
- [16] Auth0, Inc. (2024). *Introduction to JSON Web Tokens*. Recuperado de <https://jwt.io/introduction>
- [17] Tailwind Labs. (2024). *Tailwind CSS Documentation*. Recuperado de <https://tailwindcss.com/docs>
- [18] pmndrs. (2024). *Zustand Documentation*. Recuperado de <https://docs.pmnd.rs/zustand>
- [19] Evan You et al. (2024). *Vite Documentation*. Recuperado de <https://vitejs.dev>
- [20] W3C. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. Recuperado de <https://www.w3.org/TR/WCAG21>

- [21] Stack Overflow. (2024). *Developer Survey 2024*. Recuperado de <https://survey.stackoverflow.co/2024>

Apéndice A

Anexo A: Referencia completa de endpoints de la API

Método	Endpoint	Auth	Descripción
POST	/api/auth/register	No	Registrar nuevo usuario
POST	/api/auth/login	No	Iniciar sesión
GET	/api/auth/google	No	Inicio OAuth con Google
GET	/api/auth/google/callback	No	Callback OAuth Google
GET	/api/users	Sí	Listar usuarios con filtros
GET	/api/users/:id	Sí	Obtener perfil de usuario
PUT	/api/users/:id	Sí	Actualizar perfil
GET	/api/posts	Sí	Feed paginado
POST	/api/posts	Sí	Crear publicación
DELETE	/api/posts/:id	Sí	Eliminar publicación propia
POST	/api/posts/:id/like	Sí	Toggle like
GET	/api/comments/:postId	Sí	Obtener comentarios de un post
POST	/api/comments/:postId	Sí	Añadir comentario
DELETE	/api/comments/:id	Sí	Eliminar comentario propio
GET	/api/connections	Sí	Listar conexiones del usuario
POST	/api/connections/:id	Sí	Enviar solicitud de conexión
PUT	/api/connections/:id	Sí	Aceptar o rechazar solicitud
GET	/api/conversations	Sí	Listar conversaciones
GET	/api/conversations/:id	Sí	Mensajes de una conversación
POST	/api/conversations	Sí	Crear o abrir conversación
GET	/api/notifications	Sí	Notificaciones del usuario
PUT	/api/notifications/read	Sí	Marcar como leídas
GET	/api/jobs	Sí	Listar ofertas de empleo
POST	/api/jobs	Sí	Publicar oferta (empresa)
DELETE	/api/jobs/:id	Sí	Eliminar oferta propia

Tabla A.1: Referencia completa de endpoints de la API REST de FPConnect

Apéndice B

Anexo B: Ejemplo de uso de la API con curl

```
1 # 1. Registrar un alumno
2 curl -X POST https://api.fpconnect.app/api/auth/register \
3   -H "Content-Type: application/json" \
4   -d '{
5     "email": "maria@example.com",
6     "password": "MiPass2024!",
7     "role": "ALUMNO",
8     "name": "Maria Garcia"
9   }'
10 # Respuesta: { "success": true, "data": { "token": "eyJ...", "user": {...} } }
11
12 # 2. Crear una publicacion
13 curl -X POST https://api.fpconnect.app/api/posts \
14   -H "Authorization: Bearer eyJ..." \
15   -H "Content-Type: application/json" \
16   -d '{ "content": "Empiezo mis practicas en empresa tech!" }'
17
18 # 3. Buscar empresas en Sevilla
19 curl "https://api.fpconnect.app/api/users?role=EMPRESA&location=Sevilla" \
20   -H "Authorization: Bearer eyJ..."
21
22 # 4. Enviar solicitud de conexion al usuario con id "uuid-456"
23 curl -X POST https://api.fpconnect.app/api/connections/uuid-456 \
24   -H "Authorization: Bearer eyJ..."
```

Listing B.1: Flujo completo de registro, login y primer post con curl